LAPORAN PRATIKUM
STRUKTUR DATA
“STACK”

Dosen Pengampu:

DR. Wahyudi. .S.T.M.T

Asisten Lab:

Jovantri Immanuel Gulo

Disusun Oleh:

Marcello Bayu Denar Widjaksono

2511532011

Fakultas Teknologi Informasi

Departemen Informatika

Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum mata kuliah Struktur Data, dengan judul: “STACK”.

Pada kesempatan ini, Penulis mengucapkan terima kasih Bapak DR. Wahyudi. .S.T.M.T yang telah memberikan tugas laporan pratikum. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari sempurna dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis mengharapkan segala bentuk saran dan masukan serta kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap Laporan ini dapat memberikan manfaat bagi perkembangan dunia pendidikan.

DAFTAR ISI

KATA PENGANTAR……..i

DAFTAR ISI……ii

BAB I PENDAHULUAN 3

1.1 Latar Belakang 3

1.2 Tujuan 3

1.3 Manfaat Pratikum 3

BAB II PEMBAHASAN 3

2.1 Pratikum Struktur Data ..... **Error! Bookmark not defined.**

BAB III KESIMPULAN……..16

3.1 Kesimpulan ...….……16

DAFTAR PUSTAKA ……...17

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan program komputer, pemahaman mendalam tentang struktur data (data structure) menjadi fondasi utama bagi para programmer. Salah satu bentuk struktur data yang paling sering digunakan adalah Abstract Data Type (ADT), yang berfungsi mengorganisir data agar mudah diakses, dimanipulasi, dan dikelola untuk meningkatkan kinerja aplikasi.

Pada bahasa pemrograman Java, implementasi ADT Stack dapat dilakukan melalui kelas Stack yang tersedia dalam paket java.util. Stack merupakan struktur data linear yang mengimplementasikan prinsip LIFO (Last In First Out), di mana elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Perbedaan mendasar antara Stack dengan struktur data linear lainnya terletak pada aturan akses elemen: Stack hanya mengizinkan operasi penambahan dan penghapusan pada satu ujung (top), sedangkan struktur lain seperti Array atau List memungkinkan akses indeks secara acak. Karakteristik ini membuat Stack sangat efisien dan banyak diterapkan dalam berbagai skenario nyata, seperti manajemen pemanggilan fungsi (call stack), fitur undo/redo pada aplikasi, validasi tanda kurung, serta evaluasi ekspresi matematika.

1.2 Tujuan Pratikum

1. Menjelaskan konsep dan karakteristik Stack sebagai implementasi struktur data ADT dengan prinsip LIFO dalam Java.
2. Mengidentifikasi metode-metode utama Stack (seperti push, pop, peek, isEmpty, dan size) beserta mekanisme kerjanya.

1.3 Manfaat Pratikum

1. Meningkatkan pemahaman konsep struktur data bertipe LIFO melalui penggunaan Stack dalam bahasa pemrograman Java.

2. Mampu menggunakan metode-metode utama Stack untuk menyelesaikan permasalahan komputasi yang relevan.

BAB II
PEMBAHASAN

2.1 Pratikum Striktur Data

```java
package pekan3_2511532011;
import java.util.Stack;
import java.util.Scanner;


public class NilaiMaksimum_2511532011 {
	public static int max_2011(Stack<Integer> s_2011) {
		Stack<Integer> backup_2011 = new Stack<Integer>();
		int maxValue_2011 = s_2011.pop();
		backup_2011.push(maxValue_2011);

		while ( !s_2011.isEmpty()) {
			int next_2011 = s_2011.pop();
			backup_2011.push(next_2011);
			maxValue_2011 = Math.max(maxValue_2011, next_2011);
		}

		while (!backup_2011.isEmpty()) {
			s_2011.push(backup_2011.pop());
		}
		return maxValue_2011;
	}

	public static void main (String []args) {
		Stack<Integer> s_2011 = new Stack<Integer>();
		s_2011.push(70);
		s_2011.push(12);
		s_2011.push(20);

		System.out.println("isi Stack " + s_2011);
		System.out.println("Stack teratas " + s_2011.peek());
		System.out.println("Nilai teratas " + max_2011(s_2011));

	}
}
```

Pembahasan Class NilaiMaksimum:

1. Deklarasi Package dan Import

   package pekan3_2511532011;: Menetapkan namespace kelas agar terorganisir dalam struktur folder proyek.

   import java.util.Stack;: Mengimpor kelas Stack untuk digunakan sebagai struktur data LIFO (Last In First Out) penyimpanan nilai integer.

   import java.util.Scanner;: Mengimpor kelas Scanner (meskipun pada kode ini belum digunakan, umumnya disediakan sebagai template untuk menangani input pengguna dari keyboard).

2. Deklarasi Kelas

   public class NilaiMaksimum_2511532011: Mendefinisikan kelas publik yang berfungsi sebagai wadah utama program untuk mencari dan mengembalikan nilai maksimum dari sebuah Stack.

3. Metode max_2011

   public static int max_2011(Stack<Integer> s_2011): Metode statis yang menerima objek Stack sebagai parameter dan mengembalikan nilai integer tertinggi.

   Stack<Integer> backup_2011 = new Stack<Integer>();: Membuat stack cadangan untuk menyimpan elemen sementara guna menjaga urutan asli stack input.

   int maxValue_2011 = s_2011.pop();: Mengeluarkan elemen pertama dari stack dan menjadikannya nilai maksimum awal.

   backup_2011.push(maxValue_2011);: Menyimpan elemen tersebut ke stack cadangan agar data tidak hilang selama proses pencarian.

   while (!s_2011.isEmpty()) { ... }: Loop yang berjalan selama stack utama masih berisi data. Di dalamnya, s_2011.pop() mengambil elemen berikutnya, backup_2011.push() menyimpannya ke cadangan, dan Math.max() membandingkan serta memperbarui maxValue_2011.

   while (!backup_2011.isEmpty()) { s_2011.push(backup_2011.pop()); }: Memindahkan kembali seluruh elemen dari stack cadangan ke stack asli sehingga kondisi stack kembali persis seperti sebelum pemanggilan metode.

   return maxValue_2011;: Mengembalikan hasil nilai maksimum yang telah ditemukan ke pemanggil metode.

4. Metode main (Inisialisasi dan Pengisian Stack)

   public static void main(String[] args): Titik eksekusi utama program Java.

   Stack<Integer> s_2011 = new Stack<Integer>();: Menginisialisasi objek stack kosong yang akan menampung bilangan bulat selama runtime.

s_2011.push(70); s_2011.push(12); s_2011.push(20);: Menambahkan tiga nilai integer ke dalam stack secara berurutan. Sesuai prinsip LIFO, elemen teratas menjadi 20, diikuti 12, dan 70 berada di dasar.

5. Metode main (Pencetakan dan Pemanggilan Fungsi)

   System.out.println("isi Stack " + s_2011);: Mencetak representasi string dari seluruh elemen stack ke konsol menggunakan metode toString() bawaan Java.

   System.out.println("Stack teratas " + s_2011.peek());: Menggunakan metode peek() untuk membaca elemen paling atas tanpa menghapusnya dari stack, lalu mencetak nilainya.

   System.out.println("Nilai teratas " + max_2011(s_2011));: Memanggil metode max_2011 dengan melewatkan stack sebagai argumen. Hasil nilai maksimum yang dikembalikan langsung dicetak ke konsol, sementara stack secara otomatis dikembalikan ke urutan awalnya oleh logika di dalam metode.

```java
package pekan3_2511532011;
import java.util.ArrayList;

class Siswa_2011 {
	String nama_2011;
	int nim_2011;

	public Siswa_2011(String nama, int nim) {
		this.nama_2011 = nama;
		this.nim_2011 = nim;
	}

	public String toString() {
		return "Nim: " + nim_2011 + ", Nama: " + nama_2011;
	}
}

public class SiswaStack_2511532011 {
	private ArrayList<Siswa_2011> stack_2011;

	public SiswaStack_2511532011() {
		stack_2011 = new ArrayList<>();
	}

	public void push_2011(Siswa_2011 mhs) {
		stack_2011.add(mhs);
	}

	public Siswa_2011 pop_2011() {
		if (!isEmpty_2011()) {
			return stack_2011.remove(stack_2011.size()-1);
		}
		return null;
	}

	public Siswa_2011 peek_2011() {
		if (!isEmpty_2011()) {
			return stack_2011.get(stack_2011.size() - 1);
		}
		return null;
	}

	public boolean isEmpty_2011() {
		return stack_2011.isEmpty();
	}
```

```java
        public void tampilkanSiswa_2011() {
                for ( int i = stack_2011.size() - 1; i >= 0; i--) {
                        System.out.println(stack_2011.get(i));
                }

        }

        public static void main (String [] args) {
                SiswaStack_2511532011 studentStack_2011 = new SiswaStack_2511532011();

                Siswa_2011 mhs1 = new Siswa_2011("Ali", 1);
                Siswa_2011 mhs2 = new Siswa_2011("Boby", 2);
                Siswa_2011 mhs3 = new Siswa_2011("charles", 3);

                studentStack_2011.push_2011(mhs1);
                studentStack_2011.push_2011(mhs2);
                studentStack_2011.push_2011(mhs3);

                System.out.println("Siswa di dalam stack:");
                studentStack_2011.tampilkanSiswa_2011();

                System.out.println("Siswa teratas " + studentStack_2011.peek_2011());
                System.out.println("Mengeluarkan siswa teratas dari stack" + studentStack_2011.pop_2011());
                System.out.println("daftar siswa setelah di pop :");
                studentStack_2011.tampilkanSiswa_2011();
        }
}
```

Pembahasan Class Siswa dan SiswaStack:

1. Deklarasi Package dan Import

   package pekan3_2511532011;: Menetapkan namespace kelas agar terorganisir dalam struktur folder proyek.

   import java.util.ArrayList;: Mengimpor kelas ArrayList untuk digunakan sebagai struktur data dinamis yang menjadi fondasi implementasi Stack secara manual.

2. Deklarasi Kelas Siswa_2011

   class Siswa_2011: Mendefinisikan kelas entitas sederhana (default access) yang berfungsi sebagai blueprint objek data siswa dengan atribut identitas dasar.

   Atribut, Konstruktor, dan Metode toString Siswa_2011

   String nama_2011; int nim_2011;: Mendeklarasikan variabel instance untuk menyimpan nama dan nomor induk siswa.

   public Siswa_2011(String nama, int nim): Konstruktor yang menerima parameter dan menginisialisasi atribut menggunakan kata kunci this untuk membedakan scope variabel lokal dan instance.

   public String toString(): Mengoverride metode bawaan kelas Object untuk memformat objek Siswa menjadi string yang terstruktur. Metode ini dipanggil secara otomatis saat objek dicetak menggunakan System.out.println().

3. Deklarasi Kelas Utama & Inisialisasi Penyimpanan

   public class SiswaStack_2511532011: Mendefinisikan kelas publik utama yang mengemas logika operasi Stack (LIFO) menggunakan komposisi ArrayList.

   private ArrayList<Siswa_2011> stack_2011;: Mendeklarasikan variabel private sebagai wadah internal penyimpanan elemen stack.

   public SiswaStack_2511532011(): Konstruktor yang menginisialisasi ArrayList kosong saat objek stack pertama kali dibuat, mempersiapkan memori untuk operasi push.

4. Metode Operasi Stack Dasar (push, pop, peek, isEmpty)

   public void push_2011(Siswa_2011 mhs): Menerima objek siswa dan menambahkannya ke akhir ArrayList menggunakan .add(), yang secara otomatis menjadi elemen teratas stack.

   public Siswa_2011 pop_2011(): Mengecek kondisi stack melalui isEmpty_2011(). Jika tidak kosong, .remove(stack_2011.size()-1) menghapus dan mengembalikan elemen terakhir. Jika kosong, mengembalikan null untuk mencegah IndexOutOfBoundsException.

   public Siswa_2011 peek_2011(): Mirip dengan pop, tetapi menggunakan .get(stack_2011.size() - 1) untuk membaca referensi elemen teratas tanpa mengubah ukuran atau isi ArrayList.

   public boolean isEmpty_2011(): Mendelegasikan pengecekan status ke .isEmpty() milik ArrayList, mengembalikan true jika tidak ada elemen tersimpan.

5. Metode tampilkanSiswa_2011

   for (int i = stack_2011.size() - 1; i >= 0; i--): Menggunakan loop descending (mundur) dari indeks terakhir menuju 0 agar pencetakan mengikuti urutan visual stack (teratas ke terbawah).

   System.out.println(stack_2011.get(i));: Mengakses elemen berdasarkan indeks iterasi dan secara implisit memanggil metode toString() dari objek Siswa_2011 untuk menampilkan detail data ke konsol.

6. Metode main (Inisialisasi & Pengisian Data)

SiswaStack_2511532011 studentStack_2011 = new SiswaStack_2511532011();: Membuat instance stack kustom yang siap menampung objek siswa selama program berjalan.

Siswa_2011 mhs1 = ... mhs3 = ...: Membuat tiga instance objek siswa dengan data identitas yang berbeda-beda.

studentStack_2011.push_2011(mhs1)...: Memasukkan ketiga objek ke dalam stack secara berurutan. Sesuai prinsip LIFO, mhs3 (Charles) menempati posisi teratas (index terakhir).

7. Metode main (Eksekusi Operasi & Output)

studentStack_2011.tampilkanSiswa_2011();: Memanggil metode kustom untuk mencetak seluruh isi stack dari teratas ke terbawah.

studentStack_2011.peek_2011(): Mengambil dan menampilkan referensi objek teratas tanpa mengurangi jumlah elemen di dalam stack.

studentStack_2011.pop_2011(): Mengeluarkan (menghapus) objek teratas dari stack dan menyisipkannya langsung ke dalam string output konsol.

studentStack_2011.tampilkanSiswa_2011();: Memanggil metode tampilkan kembali untuk memverifikasi bahwa elemen teratas telah berhasil dihapus dan struktur stack telah diperbarui.

```java
package pekan3_2511532011;

public class StackArray_2511532011 {
	static final int MAX_2011 = 1000;
	int top_2011;
	int a_2011[] = new int [MAX_2011];

	boolean isEmpty_2011() {return (top_2011 < 0);}

	StackArray_2511532011() {top_2011 = -1 ;}

	boolean push_2011(int x) {
		if ( top_2011 >= (MAX_2011 - 1)) {
			System.out.println("Stack Overflow");
			return false;
		}
		else {
			a_2011[++top_2011] = x;
			System.out.println(x + "dimasukkan ke dalam stack");
			return true;
		}
	}

	int pop_2011() {
		if ( top_2011 < 0) {
			System.out.println("Stack Overflow");
			return 0;
		}
		else {
			int x_2011 = a_2011[top_2011--];
			return x_2011;
		}
	}

	int peek_2011() {
		if (top_2011 < 0) {
```

```
                        System.out.println("Stack underflow");
                        return 0;
                }
                else {
                        int x_2011 = a_2011[top_2011];
                        return x_2011;
                }
        }

        void print_2011() {
                for ( int i = top_2011; i > -1; i--) {
                        System.out.print (" "+ a_2011[i]);
                }
        }
}
```

Pembahasan Class StackArray:

1. Deklarasi Kelas dan Atribut Penyimpanan

   public class StackArray_2511532011: Mendefinisikan kelas publik yang berfungsi sebagai implementasi manual struktur data Stack berbasis array statis.

   static final int MAX_2011 = 1000;: Menetapkan kapasitas maksimum stack sebagai konstanta final, membatasi alokasi memori array agar bersifat fixed size dan tidak dapat berubah selama runtime.

   int a_2011[] = new int[MAX_2011];: Menginisialisasi array primitif integer sebagai media penyimpanan fisik elemen-elemen stack di memori heap.

   int top_2011;: Variabel penunjuk (index pointer) yang bertugas melacak posisi elemen teratas dalam array secara real-time.

2. Konstruktor dan Metode isEmpty_2011

   StackArray_2511532011() { top_2011 = -1; }: Konstruktor default yang mengatur nilai awal top_2011 menjadi -1, menandakan kondisi stack kosong saat objek pertama kali diinstansiasi.

   boolean isEmpty_2011() { return (top_2011 < 0); }: Mengecek status kekosongan stack dengan membandingkan nilai penunjuk. Mengembalikan true jika tidak ada elemen yang tersimpan (indeks di bawah 0).

3. Metode push_2011

   boolean push_2011(int x): Menerima parameter integer dan bertugas menambahkan elemen baru ke puncak stack.

   if (top_2011 >= (MAX_2011 - 1)): Validasi kapasitas array untuk mencegah Stack Overflow. Jika penunjuk sudah mencapai batas maksimum indeks array, operasi penambahan dibatalkan.

a_2011[++top_2011] = x;: Menggunakan pre-increment pada top_2011 terlebih dahulu, kemudian menempatkan nilai x pada indeks yang baru saja ditingkatkan.

System.out.println(...) & return true;: Memberikan notifikasi keberhasilan ke konsol dan mengembalikan status boolean true ke pemanggil metode.

4. Metode pop_2011

   int pop_2011(): Mengeluarkan dan mengembalikan nilai elemen teratas dari stack. if (top_2011 < 0): Validasi kekosongan stack. (Catatan: Pesan error di kode tertulis "Stack Overflow", namun secara terminologi teknis seharusnya "Stack Underflow"). Jika kondisi kosong terpenuhi, metode mengembalikan nilai default 0. int x_2011 = a_2011[top_2011--];: Menyimpan nilai pada indeks top_2011 ke variabel lokal, lalu menggunakan post-decrement untuk menurunkan penunjuk ke posisi sebelumnya secara otomatis.return x_2011;: Mengembalikan data yang berhasil dikeluarkan dari tumpukan.

5. Metode peek_2011

   int peek_2011(): Membaca nilai elemen teratas tanpa melakukan modifikasi atau pengurangan pada struktur stack.

   if (top_2011 < 0): Mengecek validitas akses data. Jika stack kosong, mencetak peringatan "Stack underflow" dan mengembalikan 0 sebagai nilai fallback.

   int x_2011 = a_2011[top_2011];: Mengakses langsung nilai array pada indeks top_2011 tanpa mengubah nilai penunjuk itu sendiri, menjaga integritas data tetap utuh.

   return x_2011;: Mengembalikan referensi nilai puncak untuk keperluan inspeksi data.

6. Metode print_2011

   void print_2011(): Bertugas menampilkan seluruh isi stack secara visual ke konsol.

for (int i = top_2011; i > -1; i--): Menggunakan loop descending yang dimulai dari indeks teratas (top_2011) dan berjalan mundur hingga indeks 0, sesuai prinsip visualisasi tumpukan LIFO.

System.out.print(" " + a_2011[i]);: Mencetak setiap elemen diikuti spasi secara horizontal, memungkinkan pengguna melihat urutan data dari puncak ke dasar stack dalam satu ba

```java
package pekan3_2511532011;

public class StackArrayDriver_2511532011 {
        public static void main (String []args) {
                StackArray_2511532011 s_2011 = new StackArray_2511532011();

                s_2011.push_2011(10);
                s_2011.push_2011(20);
                s_2011.push_2011(30);
                System.out.println(s_2011.pop_2011() + "dikeluarkan dari stack ");
                System.out.println("Elemen teratas adalah " + s_2011.peek_2011());
                System.out.print("Element pada stack");
                s_2011.print_2011();
        }
}
```

Pembahasan Class Mahasiswa_2511532011:

1. Deklarasi Package dan Import

   package pekan3_2511532011;: Menetapkan namespace kelas agar terorganisir dalam struktur folder proyek. (Tidak terdapat pernyataan import karena kelas ini hanya memanfaatkan tipe data primitif Java serta mengacu pada kelas StackArray_2511532011 yang berada dalam package yang sama).

2. Deklarasi Kelas

   public class StackArrayDriver_2511532011: Mendefinisikan kelas publik yang berfungsi sebagai driver atau pengendali utama alur program untuk menguji dan memvalidasi implementasi Stack berbasis array yang telah dibuat sebelumnya. Metode main (Inisialisasi Objek) public static void main(String[] args): Titik eksekusi utama program Java yang dipanggil otomatis oleh JVM saat runtime. StackArray_2511532011 s_2011 = new StackArray_2511532011();: Menginstansiasi objek stack kustom, memicu konstruktor untuk menginisialisasi array berkapasitas 1000 dan mengatur penunjuk top_2011 menjadi -1 (menandakan stack dalam kondisi kosong siap digunakan).

3. Metode main (Operasi Push)

   s_2011.push_2011(10); s_2011.push_2011(20); s_2011.push_2011(30);: Memanggil metode push secara berurutan untuk menyisipkan tiga nilai integer ke dalam stack. Sesuai prinsip LIFO (Last In First Out), 10 berada di dasar, 20 di tengah, dan 30 menempati posisi teratas dengan indeks 2.

   Metode main (Operasi Pop)

System.out.println(s_2011.pop_2011() + "dikeluarkan dari stack ");: Memanggil metode pop yang secara otomatis mengambil, menghapus, dan mengembalikan elemen teratas (30). Nilai kembalian langsung digabung dengan string output, sementara variabel top_2011 diturunkan ke indeks 1, memperbarui status internal stack.

4. Metode main (Operasi Peek)

   System.out.println("Elemen teratas adalah " + s_2011.peek_2011());: Memanggil metode peek untuk membaca nilai elemen teratas saat ini (20) tanpa melakukan penghapusan atau modifikasi pada array. Digunakan sebagai verifikasi status stack setelah operasi pop berhasil dijalankan.

5. Metode main (Pencetakan Visual & Terminasi)

   System.out.print("Element pada stack");: Mencetak label teks ke konsol menggunakan print (tanpa newline) agar output lanjutan tampil dalam baris yang sama. s_2011.print_2011();: Memanggil metode kustom yang melakukan iterasi mundur dari top_2011 ke 0, mencetak sisa elemen (20 dan 10) secara horizontal untuk memvisualisasikan struktur stack akhir. (Penutup Blok): Menandai akhir metode main dan kelas driver. JVM menyelesaikan seluruh eksekusi I/O, lalu mengakhiri program secara otomatis. Memori array dan objek dikelola oleh Garbage Collector Java tanpa memerlukan penutupan resource manual.

```java
package pekan3_2511532011;
import java.util.Stack;
import java.util.Scanner;

public class StackPostfix_2511532011 {
	public static int postfixEvaluate_2011(String expression) {
		Stack<Integer> s_2011 = new Stack<Integer>();
		Scanner input_2011 = new Scanner(expression);

		while ( input_2011.hasNext()) {

			if (input_2011.hasNextInt()) {
				s_2011.push(input_2011.nextInt());
			}
			else {
				String operator_2011 = input_2011.next();
				int operand2_2011 = s_2011.pop();
				int operand1_2011 = s_2011.pop();

				if (operator_2011.equals("+")) {
					s_2011.push(operand1_2011 + operand2_2011);
				} else if (operator_2011.equals("-")) {
					s_2011.push(operand1_2011 - operand2_2011);
				} else if (operator_2011.equals("*")) {
					s_2011.push(operand1_2011 * operand2_2011);
				} else if (operator_2011.equals("/")) {
					s_2011.push(operand1_2011 / operand2_2011);
				}
			}
		}

		input_2011.close();
		return s_2011.pop();
	}

	public static void main(String [] args) {
		System.out.println("Hasil postfix= " + postfixEvaluate_2011("5 2 4 * + 7 -"));
	}
}
```

Pembahasan Class StackPostFix:

1. Deklarasi Package dan Import

   package pekan3_2511532011;: Menetapkan namespace kelas agar terorganisir dalam struktur folder proyek.

   import java.util.Stack;: Mengimpor kelas Stack untuk digunakan sebagai struktur data LIFO (Last In First Out) dalam menyimpan operand sementara selama proses evaluasi ekspresi matematika.

   import java.util.Scanner;: Mengimpor kelas Scanner untuk mem-parse string ekspresi input menjadi token-token terpisah (angka atau operator) berdasarkan spasi.

2. Deklarasi Kelas

   public class StackPostfix_2511532011: Mendefinisikan kelas publik yang berfungsi sebagai utilitas statis untuk mengevaluasi ekspresi matematika dalam notasi postfix (Reverse Polish Notation)

3. Metode postfixEvaluate_2011 (Inisialisasi & Parsing Token)

   public static int postfixEvaluate_2011(String expression): Metode statis yang menerima string ekspresi postfix sebagai parameter dan mengembalikan hasil perhitungan akhir dalam tipe integer.

   Stack<Integer> s_2011 = new Stack<Integer>();: Menginisialisasi objek stack kosong yang akan menampung operand (bilangan bulat) yang ditemukan selama pemindaian string.

   Scanner input_2011 = new Scanner(expression);: Membuat instance Scanner yang membaca langsung dari string expression sebagai sumber data, memungkinkan ekstraksi token secara sekuensial.

   while (input_2011.hasNext()) { ... }: Loop utama yang berjalan selama Scanner masih menemukan token yang belum diproses di dalam string.

   if (input_2011.hasNextInt()) { s_2011.push(input_2011.nextInt()); }: Mengecek apakah token berikutnya merupakan bilangan bulat. Jika ya, nilai tersebut dibaca dan langsung di-push ke stack sebagai operand penyimpanan.

4. Metode postfixEvaluate_2011 (Logika Evaluasi Operator)

   else { String operator_2011 = input_2011.next(); ... }: Blok alternatif yang menangani token non-integer, mengasumsikannya sebagai operator aritmatika (+, -, *, /).

   int operand2_2011 = s_2011.pop(); int operand1_2011 = s_2011.pop();: Mengeluarkan dua nilai teratas dari stack secara berurutan. operand2 diambil terlebih dahulu sesuai prinsip LIFO, diikuti operand1, untuk menjaga urutan operasi pengurangan dan pembagian yang benar (operand1 - operand2).

   if (operator_2011.equals("+")) { ... } else if (...): Struktur kondisional if-else yang mencocokkan string operator dengan simbol matematika yang didukung.

   s_2011.push(operand1_2011 [op] operand2_2011);: Melakukan operasi aritmatika sesuai operator yang terdeteksi, lalu men-push hasil perhitungan kembali ke stack sebagai operand baru untuk operasi berikutnya.

5. Metode postfixEvaluate_2011 (Penyelesaian & Return)

   input_2011.close();: Menutup resource Scanner setelah seluruh token selesai diproses untuk mencegah kebocoran resource (resource leak) dan mengikuti praktik pemrograman yang baik. return s_2011.pop();: Mengeluarkan dan mengembalikan satu-satunya elemen yang tersisa di stack, yang merupakan hasil akhir dari seluruh evaluasi ekspresi postfix.

6. Metode main (Eksekusi & Output)

   public static void main(String [] args): Titik eksekusi utama program yang dipanggil otomatis oleh JVM.

   System.out.println("Hasil postfix= " + postfixEvaluate_2011("5 2 4 * + 7 -"));: Memanggil metode evaluasi dengan ekspresi uji "5 2 4 * + 7 -". Alur evaluasi secara bertahap: 2 * 4 = 8 (stack: 5, 8), 5 + 8 = 13 (stack: 13), 13 - 7 = 6 (stack: 6). Hasil akhir 6 langsung digabungkan dengan string dan dicetak ke konsol

BAB III
KESIMPULAN

3.1 Kesimpulan

Berdasarkan pelaksanaan praktikum, dapat disimpulkan bahwa Stack merupakan implementasi struktur data linear berbasis Abstract Data Type (ADT) dalam Java yang mengadopsi prinsip LIFO (Last In First Out), menawarkan mekanisme terkontrol untuk menambahkan, mengambil, dan memeriksa elemen teratas secara efisien melalui metode seperti push(), pop(), peek(), empty(), dan search(). Dengan kompleksitas waktu O(1) untuk operasi pada elemen teratas, Stack menjadi pilihan optimal untuk pengelolaan data yang memerlukan pembalikan urutan atau pelacakan state, meskipun memerlukan perhatian khusus terhadap penggunaan wrapper class untuk tipe data primitif serta karakteristik synchronized bawaan kelas java.util.Stack yang dapat mempengaruhi efisiensi pada skenario konkuren dibandingkan alternatif modern seperti ArrayDeque.

DAFTAR PUSTAKA

[1]W3schools,“JavaDataStructur”2026[Daring].Tersediapada:https://www.w3schools.com/java/java_data_structures.asp.[Diakses:2-april-2026]